

Model-Free RL

MC vs TD vs n -step			
	MC	TD	n -step
Target	G_t	$R + \gamma V(S')$	mixed
Bias	None	Yes	Med
Var	High	Low	Med
Episodes?	Must end	Online	Online
Bootstrap?	No	Yes	Yes

On-Policy vs Off-Policy

Behavior b : collects data

Target π : being improved

On: $b = \pi$ (SARSA, MC)

Off: $b \neq \pi$ (Q-learning)

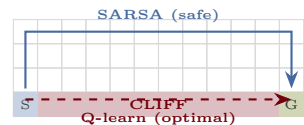
ϵ -Greedy

Greedy: $1 - \epsilon + \epsilon/|\mathcal{A}|$

Other: $\epsilon/|\mathcal{A}|$ each

Why? Pure greedy gets stuck.

SARSA vs Q-Learning



SARSA: cautious (on-policy)

Q-learn: optimal (off-policy)

Bootstrapping

Using estimates to update estimates.

TD: yes (biased, fast). MC: no (unbiased, slow).

n -Step & TD(λ)

$n=1$: TD(0). $n=\infty$: MC. Best ≈ 5 .

TD(λ): weighted avg of all n -step.

$\lambda=0$: TD(0). $\lambda=1$: MC.

Why Q not V ?

No model $\Rightarrow$ can't derive π from V .

Q : just take $\arg \max_a Q(s, a)$.

Deep Learning

Neural Networks
$f(\mathbf{x}) = \mathbf{W}^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)}$
Non-linear activations required!
Without: $\mathbf{W}_2 \mathbf{W}_1 \mathbf{x} = \mathbf{W}' \mathbf{x}$ (linear)

Activations

Sigmoid: $\frac{1}{1+e^{-x}}$, out (0, 1), vanish grad

Tanh: out $(-1, 1)$

ReLU: $\max(0, x)$ — most common

Leaky ReLU: $\max(0.01x, x)$

Overfitting Fixes

Regularization ($\lambda \sum W^2$), dropout, data augmentation, early stopping

Optimizers

SGD: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla E$

Momentum: accumulate velocity

$\mathbf{v} \leftarrow \mu \mathbf{v} + \alpha \nabla E$; $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{v}$

ADAM: adaptive lr per param

ResNets & BatchNorm

Skip: out = $F(\mathbf{x}) + \mathbf{x}$

∇ : $F' + \mathbf{I}$ (≥ 1 , no vanishing!)

BN: normalize to $\mu=0, \sigma=1$

Params vs Hyperparams

Params: learned (weights, biases)

Hyper: set before (lr, layers, ϵ)

Backpropagation

Chain rule layer by layer.

Pattern: error $\times$ weight $\times \sigma' \times$ input

Errors flow backward through network.

Universal Approx. Theorem

Enough hidden units $\rightarrow$ approx *any* continuous function. Does NOT guarantee training converges.

PyTorch

Training Loop (memorize!)
1. <code>optimizer.zero_grad()</code> clear old gradients
2. <code>out = nnnet(x)</code> forward pass
3. <code>loss = criterion(out, y)</code>
4. <code>loss.backward()</code> compute $\nabla_{\theta} \mathcal{L}$
5. <code>optimizer.step()</code> $\theta \leftarrow \theta - \alpha \nabla$

`.detach()` = stop gradient flow

`loss.item()` = scalar (no mem leak)

`.train()` = dropout ON

`.eval()` = dropout OFF

`torch.save(nnnet.state_dict())`

= save params only

Gradient Checking

$$\frac{\partial f}{\partial \theta_i} \approx \frac{f(\theta_i + h) - f(\theta_i)}{h}, h \approx 10^{-5}$$

DQN

Three DQN Innovations
1. Target network θ^- : freeze target for C steps
2. Experience replay : store (s, a, r, s') , sample random batches (breaks correlation)
3. Huber loss : clips grads for large TD errors

! Removing *either* replay OR target net $\rightarrow$ catastrophic failure

DQN Improvements

Double Q: one net picks action, other evaluates (fix max bias)

Dueling: $q = v(s) + A(s, a)$
separate state & action value

Prioritized replay: sample large-error transitions more

Rainbow: all combined

Agent57: beats all 57 Atari

Key Equations

Update Rules
MC : $V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$
TD(0) : $V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[R + \gamma V(S') - V(S_t)]}_{\text{target}}$
SARSA (on-policy): $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$
Q-learn (off-policy): $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_{a'} Q(S', a) - Q(S, A)]$
DQN Target : $y = R + \gamma \max_{a'} q_{\theta^-}(S', a')$ $L = \frac{1}{2} (y - q_{\theta}(S, A))^2$
n-step return : $G_{t:t+n} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n})$

Important Definitions
TD error : $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$
Advantage : $A_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s)$
Sigmoid : $\sigma(z) = \frac{1}{1+e^{-z}}, \sigma' = \sigma(1-\sigma)$
Softmax : $P(y=i \mathbf{x}) = \frac{e^{\mathbf{w}_i^T \mathbf{x}}}{\sum_j e^{\mathbf{w}_j^T \mathbf{x}}}$
Cross-entropy loss : $\mathcal{L} = -\sum_i [y_i \log \hat{y}_i + (1-y_i) \log(1-\hat{y}_i)]$
Robbins-Monro : $\sum \alpha_n = \infty, \sum \alpha_n^2 < \infty$

1. MC Incremental Update

Show running average = incremental form.

Start: $V_n = \frac{1}{n} \sum_{k=1}^n G_k$
Split off n -th term:

$$\begin{aligned} V_n &= \frac{1}{n} (G_n + (n-1)V_{n-1}) \\ &= \frac{1}{n} G_n + \frac{n-1}{n} V_{n-1} \\ &= \frac{1}{n} G_n + V_{n-1} - \frac{1}{n} V_{n-1} \\ &= V_{n-1} + \frac{1}{n} (G_n - V_{n-1}) \end{aligned}$$

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$

Trick: $\frac{n-1}{n} = 1 - \frac{1}{n}$

2. ϵ -Greedy Improvement

Show $q_\pi(s, \pi'(s)) \geq v_\pi(s)$.

$$\begin{aligned} &\sum_a \pi'(a|s) q_\pi(s, a) \\ &= \frac{\epsilon}{|\mathcal{A}|} \sum_a q_\pi(s, a) \\ &\quad + (1-\epsilon) \max_a q_\pi(s, a) \end{aligned}$$

Since $\max \geq$ weighted avg, use weights $w_a = \frac{\pi(a|s) - \epsilon/|\mathcal{A}|}{1-\epsilon}$:

$$\begin{aligned} &\geq \frac{\epsilon}{|\mathcal{A}|} \sum_a q_\pi \\ &\quad + \sum_a (\pi(a|s) - \frac{\epsilon}{|\mathcal{A}|}) q_\pi \\ &= \sum_a \pi(a|s) q_\pi(s, a) \\ &= v_\pi(s) \quad \square \end{aligned}$$

By policy improvement theorem:
 $v_{\pi'}(s) \geq v_\pi(s) \quad \forall s$.

3. Sigmoid Derivative

Derive $\frac{d}{dz} \sigma(z)$.

$$\sigma(z) = (1 + e^{-z})^{-1}$$

Apply chain rule:

$$\begin{aligned} \sigma' &= -(1 + e^{-z})^{-2} \cdot (-e^{-z}) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2} \end{aligned}$$

Factor:

$$\begin{aligned} &= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} \\ &= \sigma \cdot \frac{(1 + e^{-z}) - 1}{1 + e^{-z}} \\ &= \sigma \cdot (1 - \sigma) \end{aligned}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Max at $z=0$: $\sigma' = 0.25$

→ vanishing gradients in deep nets

4. Logistic Regression Gradient

Derive $\nabla_{\mathbf{w}} \mathcal{L}$ for cross-entropy.

One sample: $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x})$

$$\ell = y \log \hat{y} + (1-y) \log(1-\hat{y})$$

Using $\sigma' = \sigma(1-\sigma)$:

$$\begin{aligned} \frac{\partial \ell}{\partial \mathbf{w}} &= y \frac{\sigma(1-\sigma)}{\sigma} \mathbf{x} + (1-y) \frac{-\sigma(1-\sigma)}{1-\sigma} \mathbf{x} \\ &= y(1-\hat{y}) \mathbf{x} - (1-y) \hat{y} \mathbf{x} \\ &= (y - \hat{y}) \mathbf{x} \end{aligned}$$

Loss = $-\ell$, so over N samples:

$$\nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{N} \sum_i (\hat{y}_i - y_i) \mathbf{x}_i$$

Key: σ' cancels the denominators!

5. Backprop (2-Layer Net)

Derive weight gradients.

$$\hat{\mathbf{y}} = \mathbf{W}^{(2)} \mathbf{h}, \quad \mathbf{h} = \sigma(\mathbf{W}^{(1)} \mathbf{x})$$

$$E = \frac{1}{2} \sum_i (y_i - \hat{y}_i)^2$$

Output layer (one chain rule):

$$\begin{aligned} \frac{\partial E}{\partial W_{ij}^{(2)}} &= \frac{\partial E}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial W_{ij}^{(2)}} \\ &= -(y_i - \hat{y}_i) \cdot h_j \end{aligned}$$

Hidden layer (two chain rules):

$$\begin{aligned} \frac{\partial E}{\partial W_{jk}^{(1)}} &= \sum_i \frac{\partial E}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial h_j} \cdot \frac{\partial h_j}{\partial W_{jk}^{(1)}} \\ &= \sum_i \underbrace{[-(y_i - \hat{y}_i)] W_{ij}^{(2)}}_{\text{backprop error}} \\ &\quad \cdot \sigma'(\mathbf{W}_j^{(1)} \mathbf{x}) \cdot x_k \end{aligned}$$

Pattern:

error $\times$ weight $\times$ σ' $\times$ input

6. DAVI Gradient

Derive the gradient of AVI loss.

Target (**treat as constant!**):

$$y = \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) v_{\theta-}(s') \right]$$

$$\text{Loss: } L(\theta) = \frac{1}{2} (y - v_\theta(s))^2$$

Do NOT differentiate y :

$$\nabla_\theta L = (y - v_\theta(s)) (-\nabla_\theta v_\theta(s))$$

Update rule:

$$\theta \leftarrow \theta + \alpha (y - v_\theta(s)) \nabla_\theta v_\theta(s)$$

y uses θ^- (frozen target net).

PyTorch: `.detach()` or `no_grad()`

7. Linear Regression Closed Form

Derive $\mathbf{w}^*$ analytically.

$$\hat{\mathbf{y}} = \mathbf{X} \mathbf{w}$$

$$\mathcal{L} = (\mathbf{X} \mathbf{w} - \mathbf{y})^\top (\mathbf{X} \mathbf{w} - \mathbf{y})$$

Expand:

$$\mathcal{L} = \mathbf{w}^\top \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 \mathbf{w}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}$$

Set gradient to zero:

$$\nabla_{\mathbf{w}} \mathcal{L} = 2 \mathbf{X}^\top \mathbf{X} \mathbf{w} - 2 \mathbf{X}^\top \mathbf{y} = \mathbf{0}$$

Solve:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Key: $\nabla_{\mathbf{w}} (\mathbf{w}^\top \mathbf{A} \mathbf{w}) = 2 \mathbf{A} \mathbf{w}$

! Derivation Tricks:

- Split last term: $\sum_1^n = \sum_1^{n-1} + nth$
- $\sigma' = \sigma(1-\sigma)$ cancels log denoms
- $\max f \geq \sum w_a f$ (convex wts)
- Chain rule = multiply ∇ per layer
- $\nabla_{\mathbf{w}} (\mathbf{w}^\top \mathbf{A} \mathbf{w}) = 2 \mathbf{A} \mathbf{w}$
- **Never** diff target y in DAVI/DQN

DAVI vs DQN

DAVI: knows model $p(s'|s, a)$

$$y = \max_a [r + \gamma \sum_{s'} p(s'|s, a) v_{\theta-}(s)]$$

Approximates $v_*(s)$

DQN: model-free

$$y = r + \gamma \max_{a'} q_{\theta-}(s', a')$$

Approximates $q_*(s, a)$

Both use target networks.

DQN also needs replay buffer.